Source: docs\ADMIN_ACCESS.md | Export: 11 November 2026

Administrator access milestone

Current product workflow: PRODUCT_WORKFLOW.md. The dual administrator policy is unchanged. Initial/changed-user verification blocks; routine same-user verification keeps the mounted editor visible. Transient failure preserves the form with disabled actions and Retry permissions. Confirmed restriction or sign-out removes protected content. Cache-only verification is bounded and token expiry pauses actions; stale state never authorizes trusted backend operations.

Category milestone update: CATEGORIES.md documents the real category module and the current rules. It supersedes this historical milestone's blanket browser-write denial: enabled administrators can create/update categories and their paired slug reservations, and enabled categories are publicly readable. All other business modules retain their unavailable states.

Current storefront scope: STOREFRONT_REVIEW.md supersedes the former blanket preparation gate. Public browsing is available; customer accounts and transactions remain unavailable. The admin authorization mechanism below is unchanged.

Target: `parinaya-silks`, Cloud Firestore `(default)`. Entry: `https://parinayasilks.vercel.app/admin`. First approved account: `parinayasilks.in@gmail.com`.

Release status and scope

Local implementation provides Firebase email/password sign-in, session restoration, sign-out, neutral password-reset feedback, protected nested routes and access revocation. Production customer pages now permit browsing under the separate storefront-review milestone. Customer login/signup, product/order CRUD, payments, refunds and shipping execution are not connected by this milestone.

Manual release dependencies: enable/configure Firebase Authentication, create/confirm the approved existing account, deploy the reviewed restrictive rules, provision its real UID from a trusted environment, then commit/push and test the Vercel deployment. None of those live changes were performed automatically. No real UID or successful live provisioning is claimed.

The owner supplied the current rules: public read/write until **14 November 2026**. Their behavior is preserved as reference in `FIRESTORE_RULES_BASELINE.txt`. The local replacement `firestore.rules` has no such public grant. The deployed database remains governed by the existing permissive rule until the owner publishes the replacement. Merely deploying the route guard does not secure Firestore.

Changed implementation files

Files	Purpose
`src/admin/AdminAccess.tsx`, `AdminLogin.tsx`, `accessPolicy.ts`, `src/services/firebase/adminAuth.ts`	Real authentication, guarded sessions, safe destinations, server permission checks and recovery states.
`src/admin/AdminPanel.tsx`, `AdminLayout.tsx`, `AdminDashboard.tsx`	Reuse the panel shell and list screens with actual session identity and honest unconnected states.
`src/App.tsx`, `src/components/Header.tsx`, `src/views/AccountView.tsx`, `vercel.json`	Clean guarded admin routes, preparation-gate placement, customer shortcut removal and scoped hosting rewrites.
`firestore.rules`, `firebase.json`	Restrictive data access and isolated emulator configuration.
`scripts/provision-admin.ts`, `scripts/admin-provisioning.ts`	Trusted, confirmed UID provisioning and revocation; no automatic live changes.
`src/admin/accessPolicy.test.ts`, `scripts/admin-*`, `scripts/test-admin.mjs`, `scripts/firebase-test-cli.cjs`	Unit, rules and browser coverage using synthetic emulator accounts only.
`package.json`, `package-lock.json`, existing browser regression scripts and setup docs	Reproducible tools, updated production expectations and owner release instructions.

The earlier Firebase connection files remain in place; they are not replaced by a new application architecture.

Access mechanism

Access requires BOTH:

1. A valid Firebase Auth token with the literal custom claim `admin: true`.
2. The server-managed document `adminAccess/{that UID}` with `enabled: true`.

The frontend uses the actual authenticated UID/token, not an email comparison, customer profile role, localStorage flag or demo session. Only the trusted provisioning procedure creates/updates the access document or custom claim. Creating an Auth account by itself grants no administrator permission. There is no public administrator registration.

Firestore rules check the same claim and access record for private business collections. The category milestone permits validated administrator category writes and enabled-only public category reads; other business writes remain denied. Each signed-in account can get/listen to only its own access record; no client can list or modify access records. Undeclared collections remain denied. Customer account repositories are still unconnected; public product and owner-specific customer access require separate reviewed rules.

The access record provides a revocation mechanism independent of a previously issued token. The revoke procedure disables that record first, then removes the claim and revokes refresh tokens. Existing data access is denied by rules immediately after the record is disabled; the UI listener removes protected content. Claim changes alone are detected on token change, browser focus/visibility and a visible-page refresh every 60 seconds; Firebase token refresh propagation is not instantaneous. A cached/offline permission snapshot never authorizes the UI. Initial checks and refreshes show loading and have a ten-second failure deadline.

Sign-out unmounts the protected panel, removes its local UI state and closes the access-record listener. Navigating Back cannot bypass the Auth/permission guard. Permission/configuration/network failures show recovery actions and never open a mock admin panel. No sensitive backend endpoints exist in this repository, so no duplicate API was added. Future trusted endpoints must verify Firebase ID tokens and this same claim/access-record policy before sensitive operations; Firebase Admin SDK operations bypass rules and must remain trusted.

Existing screens and honest data states

The existing admin layout, colors, typography, sidebar and mobile menu are retained. Demo role toggles, fake staff identity, fixed order badge and simulated sign-out are removed. Every admin path goes through the same guard, including legacy hash links and direct clean URLs.

Area	Current behavior after administrator authorization
Dashboard	Original dashboard component has an unavailable mode: metrics use dashes, charts and recent orders indicate unconnected reporting. No fictional revenue or trends.
Orders	Existing table layout receives no production records; controls and exports are disabled under an explicit unavailable notice.
Order Details	Unavailable state; no fabricated order and no fulfilment/payment action. Existing detail editor code is retained for the next module milestone.
Products & Inventory	Existing empty table layout, disabled mutation controls; navigation opens the product-editor unavailable state.
Add / Edit Product	Unavailable state until a real catalog repository is implemented. Existing form code is retained; no sample-image or simulated save workflow is enabled in production.
Customers	Existing empty customer layout; no sample identities or order history.
Coupons & Loyalty	Unavailable state. The sample loyalty engine is not described as active. Existing editor code is retained.
Store Settings	Unavailable state; no fictional shipping values and no localStorage save/reset presented as a live store update. Existing form code is retained.

The category module now has its own protected route and Manage Categories entry from Products & Inventory. The other areas above remain unconnected business modules. Authentication readiness does not imply their mutation behavior has been released. Customer navigation contains no administrator shortcut.

Firebase Console setup (owner actions)

1. Open Firebase Console and select **parinaya-silks**. Confirm Firestore database **(default)**.
2. Authentication → Sign-in method → Email/Password: enable the password-based provider and save. No Google, phone or email-link provider is required by this task.
3. Authentication → Settings → Authorized domains: ensure `parinayasilks.vercel.app` is present. Add `localhost` if it is needed for the owner's local testing. Do not change `VITE_FIREBASE_AUTH_DOMAIN` to the Vercel hostname; retain the supplied `parinaya-silks.firebaseapp.com` value.
4. Authentication → Users: locate `parinayasilks.in@gmail.com`. If absent, use Add user and set the password privately. Do not put the password in chat, scripts, environment variables or source. Record its actual UID privately. Existing users should not be recreated.
5. Review `firestore.rules` against any real integrations before release. The supplied global development grant is replaced, not layered alongside the new conditions. No `allow true` wildcard should remain.

6. Firestore Database → `(default)` → Rules: publish the reviewed local `firestore.rules`. Alternatively, from the correct trusted CLI account, run the following explicitly targeted command yourself:

```
npx.cmd firebase deploy --only firestore:rules --project parinaya-silks
```

No rule deployment command was executed by this task. The current replacement permits enabled-only public category reads and validated administrator category writes; other private data and business mutation restrictions remain. If an external integration not present in this repository relies on the test-mode grant, review its exact access before deployment; do not retain the broad grant. See CATEGORIES.md for the current release procedure.

See Firebase password authentication and custom claims.

Trusted provisioning (owner actions)

Use Node 22+ on a trusted workstation or Cloud Shell with this checkout and installed dependencies. The provisioning SDK is a development dependency and is never imported by frontend modules. Do not run this in a customer browser, add a public provisioning endpoint, or supply Admin credentials to Vite/Vercel frontend variables.

Use an existing trusted identity with Application Default Credentials and permission to read/update Firebase Auth users/custom claims and read/write the single access record in project `parinaya-silks`. Follow the Firebase Admin setup guidance. For an authorized local Google Cloud SDK identity, ADC setup is normally:

```
gcloud auth application-default login
```

If using an already approved service-account credential instead, set `GOOGLE_APPLICATION_CREDENTIALS` to its protected file **outside this checkout** in the trusted shell only. The script rejects service-account files for another project, conflicting project variables and emulator variables. Do not generate/upload a new key merely for the frontend.

First perform the default read-only inspection:

```
npm.cmd run admin:provision
```

The script explicitly targets project `parinaya-silks`, resolves the exact approved email and prints its real UID/email for confirmation without printing credentials or custom-claim values. If the account does not exist, it stops and directs you back to Firebase Authentication user creation. If trusted credentials are unavailable, stop here; no UID is invented.

After reviewing that resolved identity, grant access explicitly:

```
npm.cmd run admin:provision -- --apply --confirm-uid ACTUAL_FIREBASE_UID --confirm-email parinayasilks.in@gmail.com
```

Replace `ACTUAL_FIREBASE_UID` with the UID displayed by the inspection, not the example test fixture UID. The script re-reads the account before writing, verifies UID/email, preserves unrelated custom claims, sets `admin: true`, and enables `adminAccess/{UID}` with trusted metadata. Rerunning the same confirmed grant is safe and does not replace other claims. Run only one role-management operator at a time; Firebase custom-claim updates are replacements, not a transactional merge across concurrent operators.

If a network error interrupts the two-service update, it reports that a partial update is possible. Repeat the same confirmed operation to reconcile it. A missing/disabled record or missing claim fails closed. After a successful grant, use **Refresh permissions**, or sign out and sign in again to obtain a fresh token.

To revoke this approved first account later, the trusted owner can run:

```
npm.cmd run admin:provision -- --action revoke --apply --confirm-uid ACTUAL_FIREBASE_UID --confirm-email parinayasilks.in@gmail.com
```

This disables the access record before removing the claim and revoking refresh tokens. The script is intentionally limited to the approved first administrator; adding more administrators requires an explicit new approval/provisioning decision.

Local and Vercel configuration

Retain the seven required public web variables from `docs/FIREBASE_SETUP.md`: `VITE_FIREBASE_API_KEY`, `VITE_FIREBASE_AUTH_DOMAIN`, `VITE_FIREBASE_PROJECT_ID`, `VITE_FIREBASE_STORAGE_BUCKET`, `VITE_FIREBASE_MESSAGING_SENDER_ID`, `VITE_FIREBASE_APP_ID`, `VITE_FIREBASE_DATABASE_ID`. Database ID is `(default)`. `.env.local` is ignored. No admin email, password, UID allowlist or server-secret variable is needed in browser configuration.

Set the same reviewed browser variables in the existing Vercel project's Production environment and Preview separately if used. Keep `npm run build` / output `dist`. Trigger a fresh build after configuration changes. Do not set `build:demo` or emulator variables for production. Do not upload provisioning credentials to Vercel for this frontend-only milestone.

`vercel.json` rewrites only `/admin` and `/admin/:path*` to `index.html`. Root customer hash navigation remains intact. `/api/*` and `/assets/*` are not swallowed by an application-wide fallback. The actual hosted rewrite behavior must be verified after the owner deploys; a local Vite preview does not establish Vercel state. See Vercel rewrites.

Local checks

```
npm.cmd run lint
npm.cmd test
npm.cmd run build
npm.cmd run test:admin
```

The last command starts local Auth (`127.0.0.1:9099`) and Firestore (`127.0.0.1:8080`) emulators, loads the local rules, and creates synthetic emulator accounts only. It runs rule tests, a Vite `admin-test` server at port 3100, a missing-configuration test server at port 3101 and a production preview at port 4200, then shuts them down. Python Playwright/Chromium and Java 21+ are required. `JAVA_HOME` is honored if valid; an already downloaded portable Java under ignored `.cache/java/runtime` is also discovered. No system Java installation is changed. The emulator JAR cache remains ignored under `.cache`.

The development-only `admin-test` mode explicitly connects to the loopback emulators. The production branch does not contain those connections. Business data reads are not triggered by the panel. Test scripts reject missing/non-loopback emulator variables before using the Admin SDK. A random synthetic test password exists only in test-process memory/environment and is never printed or persisted by the harness. The test CLI bootstrap suppresses inherited-environment debug dumps. The live provisioning script rejects emulator variables so it cannot be confused with the test fixture tools.

For manual real-account local testing after owner setup:

```
npm.cmd run dev
```

Open `http://localhost:3000/admin`. Use the privately configured account password yourself. A missing role/record shows Access Restricted; unknown/missing configuration and rules/network failures show recovery states. Do not use real customer records to test unrelated modules.

Production acceptance after owner deployment

At `https://parinayasilks.vercel.app/admin`:

1. In a private window, confirm signed-out login, accessible labels, show/hide password and neutral incorrect-credential feedback.
2. Visit `/admin/orders` and `/admin/products/new` directly while signed out. They must show login with no protected-content flash. After approved sign-in, the original internal screen should appear.
3. Refresh a nested page. Confirm restored Firebase session and a new server permission check.
4. Confirm dashboard/list/editor areas explicitly say their business data is unconnected, with no sample sales/customers/orders and no successful fake saves.
5. Test an ordinary controlled account: Access Restricted. An Auth account or matching email by itself must never open the panel.
6. Test Sign out, Back and refresh. The guarded screens must remain unavailable while signed out.
7. Test Forgot Password with controlled existing/absent addresses; do not disclose results or passwords publicly.
8. If testing revocation, use a controlled administrator and the trusted procedure. Disabling its access record must remove protected content and deny direct business reads with the old token. Restore only through the trusted owner process.
9. Check `/` shows the original storefront with its sample-catalogue notice; no customer admin link, demo checkout release or admin role toggle exists.
10. Verify direct nested refresh on Vercel, static assets and any future `/api/*` routes independently. The local rewrite configuration does not prove hosted routing before deployment.

Do not treat the access milestone as live-complete until the owner has performed the Console/rules/provisioning/deployment steps above. No live account password, real UID, trusted credential or production session was used by this implementation task.

Local validation results (15 October 2026)

- TypeScript checking and production build passed. The build reports an admin bundle size warning above 500 kB before gzip; this is not a build failure.
- All 28 unit tests passed, including expiry/claim checks, safe destinations and provisioning safeguards.
- Auth/Firestore emulator suite passed: direct data-access denial, administrator reads, blocked browser writes, role/record combinations, revocation, login/reset/session restoration, sign-out/Back, duplicate submission, mobile layout and missing-configuration recovery.
- Existing customer browser flows and intercepted Firebase browser fixtures passed.
- Existing route/viewport audit passed all 192 checks with no page errors or horizontal overflow.

- Production-build preparation and admin routing were checked through local preview only. Firebase Console settings, real account privileges, deployed rules and hosted Vercel behavior remain owner acceptance steps.